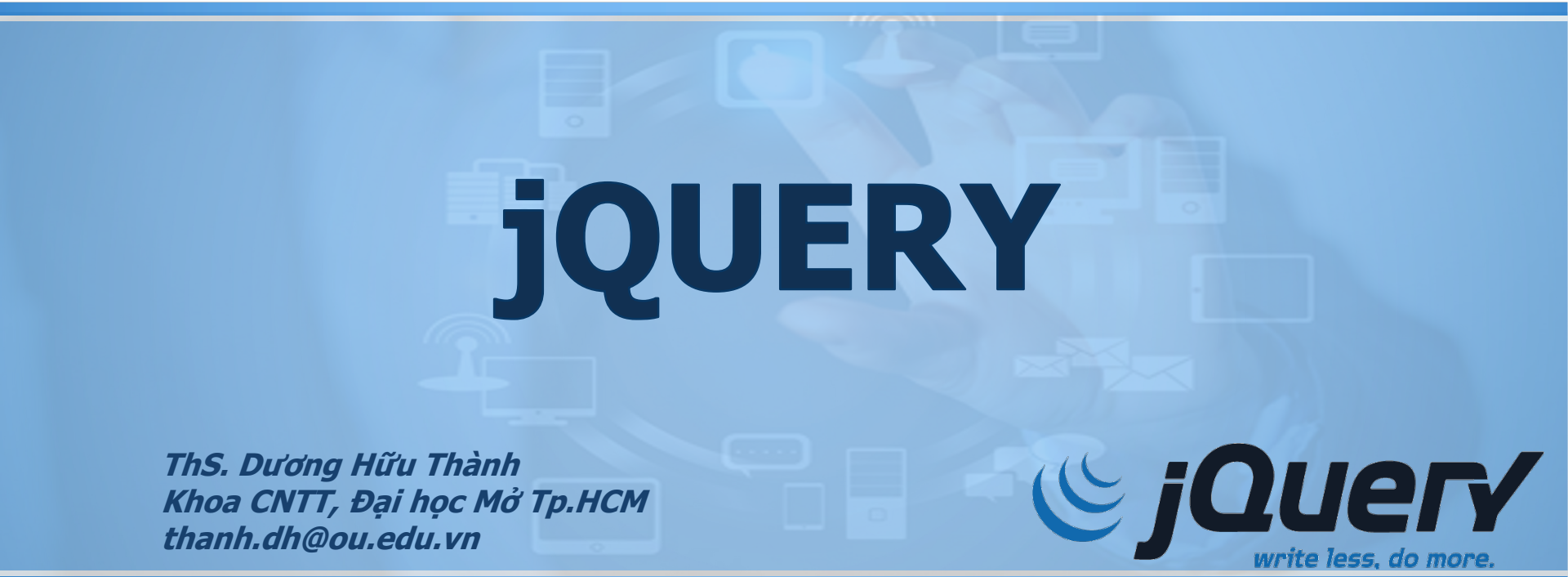


THIẾT KẾ WEB



jQUERY



ThS. Dương Hữu Thành
Khoa CNTT, Đại học Mở Tp.HCM
thanh.dh@ou.edu.vn



HTML



JS



CSS





- 1. Giới thiệu jQuery**
- 2. Đặc trưng jQuery**
- 3. Sử dụng jQuery**
- 4. Cú pháp jQuery**
- 5. Xử lý sự kiện với jQuery**
- 6. Hiệu ứng**
- 7. Thao tác với HTML DOM**

- jQuery là một **thư viện Javascript** được viết bởi John Resig năm 2006.
- jQuery làm **đơn giản hóa** thao tác với tài liệu HTML.

"Write less, do more"

- Thao tác với HTML DOM.
- Thao tác với CSS.
- Các phương thức xử lý sự kiện HTML event.
- Hiệu ứng (effects) và hoạt cảnh (animations).
- Hỗ trợ AJAX (Asynchronous Javascript And XML).
- Hỗ trợ bởi nhiều trình duyệt.

- Download thư viện jQuery tại <http://jquery.com/>
- Có hai phiên bản jQuery
 - Production: dùng cho production (được nén nhỏ gọn)
 - Development: dùng cho testing và development (không nén, có thể đọc code)

```
<head>  
<script src="jquery-3.2.1.min.js"></script>  
</head>
```

- Sử dụng jQuery thông qua các CDN (Content Delivery Network)

Google CDN

```
<script  
src="https://ajax.googleapis.com/ajax/libs/jquery/3  
.2.1/jquery.min.js"></script>
```

Microsoft CDN

```
<script  
src="https://ajax.aspnetcdn.com/ajax/jQuery/jquery-  
3.2.1.min.js"></script>
```

CDNJS CDN

```
<script src="  
https://cdnjs.cloudflare.com/ajax/libs/jquery/3.2.1  
/jquery.min.js "></script>
```

- **\$** ký hiệu để truy cập jQuery
- **selector**: chỉ định các thành phần HTML thao tác.
- **action**: thực hiện một hành động trên một thành phần HTML

```
$(selector).action();
```

- Ví dụ:
 - `$("p").show();` // hiển thị thành phần p
 - `$("#main").hide();` // ẩn thành phần có id là main
 - `$("div.color").hide();` // ẩn thành phần div có class là color

- Tất cả phương thức jQuery nên đặt trong sự kiện document ready.

```
$(document).ready(function() {  
    // ...  
});
```

```
$(function() {  
    // ...  
});
```

- Sự kiện này chỉ được thực thi khi tài liệu HTML đã được nạp đầy đủ.

The background features a stylized world map in shades of blue and purple. Overlaid on the map is a complex network of white lines connecting various points, resembling a global communication or data network. The lines are thin and create a web-like pattern across the entire image.

jQuery

XỬ LÝ SỰ KIỆN

- Cú pháp xử lý sự kiện

```
$(selector).<event>(function() {  
  
});
```

- Xử lý sự kiện với phương thức **on**

```
$(selector).on({  
  click: function() {},  
  mouseenter: function() {},  
  mouseleave: function() {}  
});
```

- Sự kiện chuột
 - click
 - dbclick
 - mouseenter
 - mouseleave
 - hover
- Sự kiện bàn phím
 - keypress
 - keydown
 - keyup
- Sự kiện form
 - submit
 - change
 - focus
 - blur
- Sự kiện window
 - load
 - resize
 - scroll
 - unload

Tham khảo: https://www.w3schools.com/jquery/jquery_ref_events.asp

JQUERY

HIỆU ỨNG

- **hide()/show()**: ẩn/hiện thành phần HTML

```
$(selector).hide([<speed>, <callback>]);  
$(selector).show([<speed>, <callback>]);
```

- **toggle()**: hide() + show() → ẩn thành phần đang hiển thị và hiển thị thành phần đang ẩn.

- Trong đó:

- <speed> là tốc độ ẩn/hiện thành phần, giá trị của nó có thể "slow", "fast", hoặc một số milli giây.
- <callback> là hàm sẽ được gọi sau khi ẩn/hiện hoàn tất.

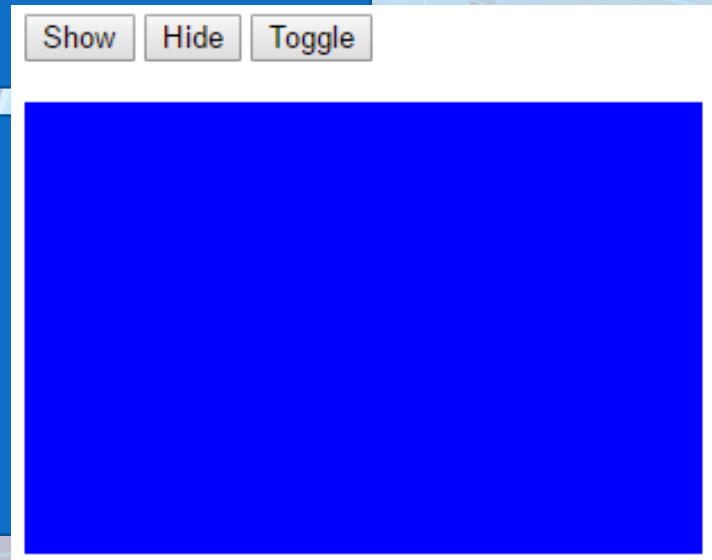
- Ví dụ: cho HTML và CSS như sau

HTML

```
<div>
  <button id="show">Show</button>
  <button id="hide">Hide</button>
  <button id="toggle">Toggle</button>
</div>
<br />
<div class="obj"></div>
```

CSS

```
div.obj {
  display: inline-block;
  background-color: blue;
  width: 300px;
  height: 200px;
}
```



- Xử lý sự kiện hide & show thành phần div khi click trên các button tương ứng bằng jQuery

```
$(document).ready(function() {  
    $("#show").click(function() {  
        $("div.obj").show();  
    });  
    $("#hide").click(function() {  
        $("div.obj").hide();  
    });  
    $("#toggle").click(function() {  
        $("div.obj").toggle();  
    });  
});
```

- **fadeOut()**: mờ dần đến khi ẩn thành phần HTML
- **fadeIn()**: hiện dần thành phần HTML
- **fadeToggle()**: fadeIn() + fadeOut()

```
$(selector).fadeIn(<speed>, <callback>);  
$(selector).fadeOut(<speed>, <callback>);  
$(selector).fadeToggle(<speed>, <callback>);
```

- **fadeTo()**: mờ dần thành phần HTML đến giá trị opacity (giá trị từ 0 → 1) chỉ định.

```
$(selector).fadeTo(<speed>, <opacity>, <callback>);
```

- **slideDown()**: trượt thành phần HTML xuống.
- **slideUp()**: trượt thành phần HTML lên.
- **slideToggle()**: slideDown() + slideUp()

```
$(selector).slideDown(<speed>, <callback>);  
$(selector).slideUp(<speed>, <callback>);  
$(selector).slideToggle(<speed>, <callback>);
```

- Phương thức **animate()** cho phép tạo các hiệu ứng tùy chỉnh.

```
$(selector).animate({<params>}, <speed>, <callback>);
```

- **<params>** (bắt buộc) chỉ định các **thuộc tính css** thực hiện hiệu ứng.
- **<speed>** (tùy chọn) chỉ định **khoảng thời gian** thực hiện hiệu ứng, có thể nhận giá trị "slow", "fast" hoặc một giá trị milli giây.
- **<callback>** (tùy chọn) chỉ định hàm được thực thi khi hiệu ứng hoàn tất.

- Ví dụ: di chuyển thành phần div sang trái

```
<button id="btnPlay">Play</button>  
<br />  
<div></div>
```

```
div {  
  display: inline-block;  
  width: 150px;  
  height: 100px;  
  background-color: blue;  
  position: relative; -----  
}
```

Mặc định position các thành phần HTML là static không thể di chuyển được nên muốn hiệu ứng thay đổi vị trí thì đầu tiên phải thiết lập position là relative, fixed, hoặc absolute

- Xử lý javascript khi click trên button Play

```
$(document).ready(function() {  
    $("#btnPlay").click(function() {  
        $("div").animate({"left": "300px"});  
    });  
});
```

Tên các thuộc tính được viết theo quy tắc **lowerCamelCase**, chẳng hạn padding-left thì viết là paddingLeft

- Các thuộc tính trong `<params>` có thể nhận các giá trị "show", "hide" hoặc "toggle".
- Có thể xác định **giá trị tương đối** từ giá trị hiện tại của thuộc tính bằng ký hiệu `+=`, `-=`
- Nếu thực hiện nhiều animate cho cùng một thành phần thì jQuery sẽ tạo một **hàng đợi** cho chúng và thực hiện từng cái một.
- Các hiệu ứng màu sắc không bao gồm trong thư viện core jQuery.

- Ví dụ: cho thẻ `div` di chuyển sang phải **200px**, sau đó thay đổi kích thước chữ là **36px** và tăng thêm chiều cao `div` là **30px**.
- HTML và CSS như sau:

HTML

```
<div id="textId">jQuery</div>
```

CSS

```
div#textId {  
    width:100px;  
    height:20px;  
    background-color:#666;  
    color:#fff;  
    position:relative;  
}
```

- Mã nguồn jQuery xử lý hiệu ứng trên

```
$(document).ready(function() {  
    var div = $("div#textId");  
    div.animate({  
        "left": "200px",  
    }, "slow");  
  
    div.animate({  
        "fontSize": "36px",  
        "height": "+=30px"  
    }, "slow");  
});
```

- Phương thức `stop()` dùng để dừng một hiệu ứng.

```
$(selector).stop(<stopAll>, <goToEnd>);
```

The background features a stylized world map in shades of blue and purple. Overlaid on the map is a complex network of white lines connecting various points, resembling a global communication or data network. The lines are thin and create a web-like pattern across the entire image.

jQuery

THAO TÁC VỚI HTML DOM

- Các phương thức cơ bản thao tác với HTML DOM
 - **text()**: lấy/thiết lập nội dung văn bản cho thành phần được chọn.
 - **html()**: lấy/thiết lập nội dung cho thành phần được chọn (bao gồm các thẻ HTML).
 - **val()**: lấy/thiết lập giá trị cho form field.
- Phương thức **attr()**: lấy/thiết lập giá trị một thuộc tính của thành phần HTML được chọn.

- Ví dụ: Tạo giao diện HTML để người dùng nhập tên và click trên button "Hello":
 - Nếu input name rỗng hiển thị dòng thông báo lỗi trong thành phần `<p>` màu đỏ (red).
 - Ngược lại hiển thị dòng chữ "Hello `<name>`" cũng trong thành phần `<p>` nhưng màu xanh (blue).

```
<input type="text" id="name"
      placeholder="Enter your name..." />
<input type="button" id="btnName" value="Hello" />
<p id="helloId"></p>
```


Please enter your name!

Hello: Thành

▪ Xử lý javascript

```
$(document).ready(function() {  
    $("#btnName").click(function() {  
        var name = $("#name").val();  
        var helloEl = $("#helloId");  
        helloEl.val("");  
        if (name == "") {  
            helloEl.attr("style", "color:red;");  
            helloEl.text("Please enter your name!");  
        } else {  
            helloEl.attr("style", "color:blue;");  
            var msg = "Hello: " + "<b>" + name + "</b>";  
            $("#helloId").html(msg);  
        }  
    });  
});
```

- Các phương thức `text()`, `html()`, `val()`, `attr()` cũng có **hàm callback**, hàm này chứa hai tham số:
 - Tham số 1: là **chỉ số** thành phần đang xét.
 - Tham số 2: **giá trị hiện tại** của thành phần đang xét.
- Ví dụ: hiển thị dòng chữ "Hello <name>" với các thẻ heading dựa trên chỉ số của thành phần đang xét.

▪ HTML

```
<input type="text" id="name"
        placeholder="Enter your name..." />
<input type="button" id="btnName" value="Hello" />
<p class="hello"></p>
<p class="hello"></p>
<p class="hello"></p>
<p class="hello"></p>
<p class="hello"></p>
```

Hello Thành

Hello Thành

Hello Thành

Hello Thành

Hello Thành

▪ Xử lý javascript

```
$(document).ready(function() {  
    $("#btnName").click(function() {  
        var name = $("#name").val();  
        $(".hello").html(function(idx, originalHtml) {  
            var t = "h" + (idx + 1)  
            var h = "<" + t + ">Hello " + name + "</" +  
t + ">";  
            return h;  
        })  
    });  
});
```

- Các phương thức thêm nội dung mới vào thành phần HTML
 - **append()**: chèn nội dung vào **cuối** thành phần được chọn.
 - **prepend()**: chèn nội dung vào **đầu** thành phần được chọn.
 - **after()**: chèn nội dung **sau** thành phần được chọn.
 - **before()**: chèn nội dung **trước** thành phần được chọn.

- Ví dụ: cho HTML như sau

```
<button id="btnAppend">Append ul</button>
<button id="btnPrepend">Prepend ul</button>
<button id="btnAfter">After ul</button>
<button id="btnBefore">Before ul</button>
<ul id="item">
  <li>Item 1</li>
  <li>Item 2</li>
</ul>
```

Append ul

Prepend ul

After ul

Before ul

- Item 1
- Item 2

```
$("#btnAppend").click(function() {  
    var idx = parseInt(Math.random() * 100);  
    $("ul").append("<li>Item " + idx + "</li>");  
});  
$("#btnPrepend").click(function() {  
    var idx = parseInt(Math.random() * 100);  
    $("ul").prepend("<li>Item " + idx + "</li>");  
});  
$("#btnAfter").click(function() {  
    var m = "<p>Random number:" + Math.random() + "</p>";  
    $("ul").after(m);  
});  
$("#btnBefore").click(function() {  
    var m = "<p>Random number:" + Math.random() + "</p>";  
    $("ul").before(m);  
});
```

Xử lý javascript khi click trên các button



- Các phương thức trên có thể thực hiện cùng lúc với nhiều thành phần.
- Có thể tạo thành phần HTML bằng **text/html**, **jquery**, **javascript DOM** như ví dụ sau:

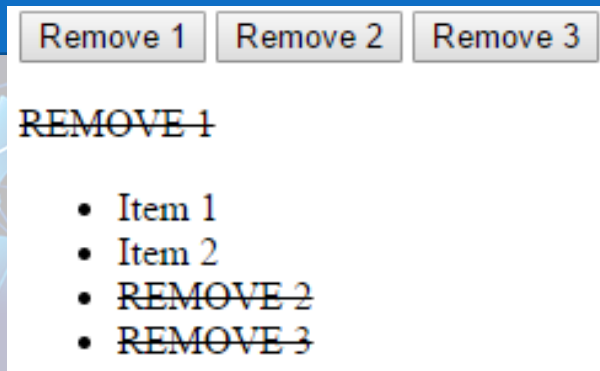
```
$("#btnAppend").click(function() {  
    var idx = parseInt(Math.random() * 100);  
  
    var e1 = "<li>Item " + idx + "</li>";  
    var e2 = $("<li></li>").text("Item " + idx);  
    var e3 = document.createElement("li");  
    e3.innerText = "Item " + idx;  
  
    $("ul").append(e1, e2, e3);  
});
```



- Các phương thức xóa thành phần HTML
 - **remove([<selector-filter>])**: xóa thành phần được chọn và các thành phần con của nó.
 - **empty()**: xóa các thành phần con của thành phần được chọn.

▪ Ví dụ:

```
<button id="rmId1">Remove 1</button>
<button id="rmId2">Remove 2</button>
<button id="rmId3">Remove 3</button>
<p id="rmDivId"><del>REMOVE 1</del></p>
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
  <li class="rm1"><del>REMOVE 2</del></li>
  <li class="rm2 del"><del>REMOVE 3</del></li>
</ul>
```





- Xử lý javascript khi click trên các button

```
$("#rmId1").click(function() {  
    // xóa thành phần có id là #rmDivId  
    $("#rmDivId").remove();  
});
```

```
$("#rmId2").click(function() {  
    // xóa thành phần li có class chứa rm1  
    $("ul>li").remove(".rm1");  
});
```

```
$("#rmId3").click(function() {  
    // xóa thành phần li có class chứa rm2 và del  
    $("ul>li").remove(".rm2, .del");  
});
```

- Các phương thức thao tác với CSS
 - **addClass()**: thêm một hoặc nhiều class mới vào thành phần được chọn.
 - **removeClass()**: xóa một hoặc nhiều class từ thành phần được chọn.
 - **toggleClass()**: kết hợp thêm và xóa class từ thành phần được chọn.
 - **css()**: lấy/thiết lập giá trị cho một thuộc tính CSS.

- Ví dụ: cho người dùng nhập first name và last name khi click Submit:
- Nếu first name hoặc last name rỗng thì thêm class error với css có viền đỏ như hình bên.
- Nếu nhập đủ 2 trường thì hiện thị full name và xóa class error khỏi hai input trên.

First Name

Last Name

First Name

Last Name

First Name

Last Name

First Name

Last Name

Your full name is **Thành Dương**

```
<table>
  <tr>
    <td>First Name</td>
    <td><input type="text" id="firstName" /></td>
  </tr>
  <tr>
    <td>Last Name</td>
    <td><input type="text" id="lastName" /></td>
  </tr>
  <tr>
    <td colspan="2" align="right">
      <button id="btnSubmit">Submit</button>
    </td>
  </tr>
  <tr>
    <td colspan="2">
      Your full name is <span id="fullName"></span>
    </td>
  </tr>
</table>
```

```
$("#btnSubmit").click(function() {  
    var check = function(el) {  
        if (el.val() == "") {  
            el.addClass("error");  
            return false;  
        }  
        el.removeClass("error");  
        return true;  
    };  
    var fn = $("#firstName");  
    var ln = $("#lastName");  
    if (check(fn) && check(ln)) {  
        var msg = fn.val() + " " + ln.val();  
        $("#fullName").text(msg);  
    }  
});
```

Xử lý khi click
trên button
Submit

```
.error {  
    border: 1px solid red;  
}
```

CSS cho lớp error

- Sử dụng phương thức `css()`

- Lấy giá trị một thuộc tính

```
$(selector).css(<property-name>);
```

- Thiết lập giá trị cho một thuộc tính

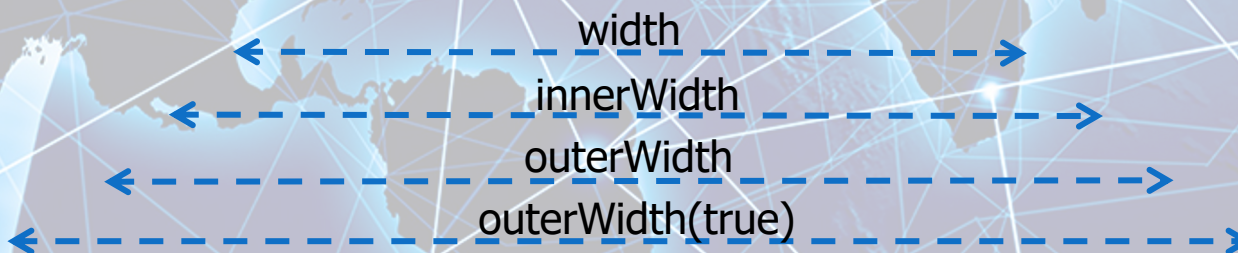
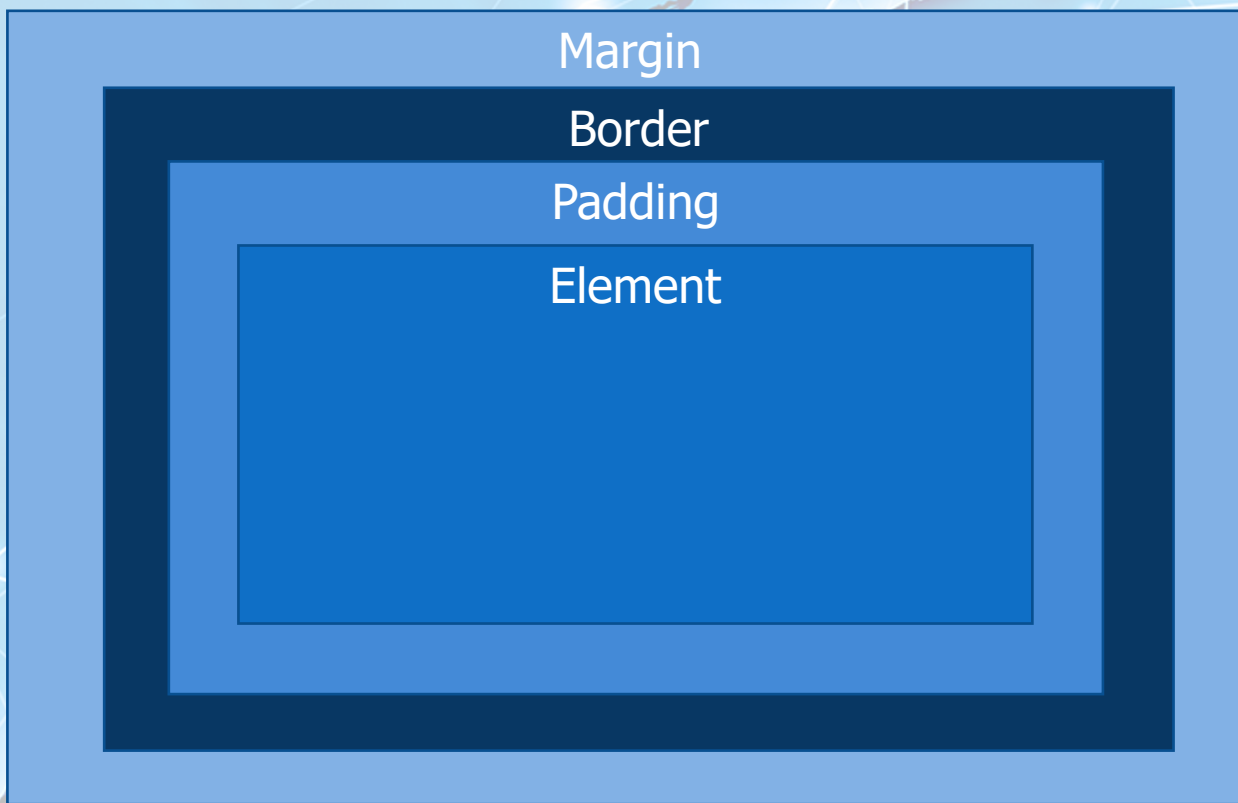
```
$(selector).css(<property-name>, <value>);
```

- Thiết lập giá trị cho nhiều thuộc tính cùng lúc

```
$(selector).css({  
    <property-name-1>: <value>,  
    ...  
    <property-name-n>: <value>  
});
```



- Các phương thức làm việc với kích thước của các thành phần HTML
 - `width()/height()`: thiết lập hay lấy chiều rộng và chiều cao các thành phần (không bao gồm padding, margin, và border).
 - `innerWidth()`=`width()`+<padding-width>
 - `innerHeight()`=`height()`+<padding-height>
 - `outerWidth()`=`innerWidth()`+<border-width>
 - `outerHeight()`=`innerHeight()`+<border-height>
 - `outerWidth(true)`=`outerWidth()`+<margin-width>
 - `outerHeight(true)`=`outerHeight()`+<margin-height>



Q&A